

$$= \begin{bmatrix} w'_{11}x_{i1} + w'_{21}x_{i2} + w'_{31}x_{i3} + w'_{41}x_{i4} \\ w'_{12}x_{i1} + w'_{22}x_{i2} + w'_{32}x_{i3} + w'_{42}x_{i4} \\ w'_{13}x_{i1} + w'_{23}x_{i2} + w'_{33}x_{i3} + w'_{43}x_{i4} \end{bmatrix} + \begin{bmatrix} b_{11} \\ b_{12} \\ b_{13} \end{bmatrix}$$

$$\Rightarrow \sigma \left(\begin{bmatrix} w'_{11}x_{i1} + w'_{21}x_{i2} + w'_{31}x_{i3} + w'_{41}x_{i4} + b_{11} \\ w'_{12}x_{i1} + w'_{22}x_{i2} + w'_{32}x_{i3} + w'_{42}x_{i4} + b_{12} \\ w'_{13}x_{i1} + w'_{23}x_{i2} + w'_{33}x_{i3} + w'_{43}x_{i4} + b_{13} \end{bmatrix} \right)$$

$$= \begin{bmatrix} 0.11 \\ 0.12 \\ 0.13 \end{bmatrix} \rightarrow \text{output of layer 1} \\ = a^{[1]} \rightarrow \text{activation of layer 1}$$

layer 2.

$$\begin{bmatrix} w^2_{11} & w^2_{12} \\ w^2_{21} & w^2_{22} \\ w^2_{31} & w^2_{32} \end{bmatrix}^T \begin{bmatrix} 0.11 \\ 0.12 \\ 0.13 \end{bmatrix} + \begin{bmatrix} b_{21} \\ b_{22} \end{bmatrix}$$

(3,2) $\rightarrow$ (2,3) (3,1) (2,1)

$$= \begin{bmatrix} w^2_{11}0.11 + w^2_{21}0.12 + w^2_{31}0.13 + b_{21} \\ w^2_{12}0.11 + w^2_{22}0.12 + w^2_{32}0.13 + b_{22} \end{bmatrix}$$

$$\Rightarrow \sigma \left(\begin{bmatrix} w^2_{11}0.11 + w^2_{21}0.12 + w^2_{31}0.13 + b_{21} \\ w^2_{12}0.11 + w^2_{22}0.12 + w^2_{32}0.13 + b_{22} \end{bmatrix} \right) = \begin{bmatrix} 0.21 \\ 0.22 \end{bmatrix} \rightarrow \text{output of layer 2} \\ = a^{[2]} \rightarrow \text{activation of layer 2}$$

layer 3

$$\begin{bmatrix} w^3_{11} \\ w^3_{21} \end{bmatrix}^T \begin{bmatrix} 0.21 \\ 0.22 \end{bmatrix} + b_{31}$$

(2,1) $\rightarrow$ (1,2) (2,1)

$$\Rightarrow \sigma \left([w^3_{11}0.21 + w^3_{21}0.22 + b_{31}] \right) = 0.31 = \hat{y}_i \rightarrow \text{final output} \\ = a^{[3]} \rightarrow \text{activation 3}$$

$$a^{[1]} = \sigma(a^{[0]}w^{[1]} + b^{[1]}) \\ a^{[2]} = \sigma(a^{[1]}w^{[2]} + b^{[2]}) \\ a^{[3]} = \sigma(a^{[2]}w^{[3]} + b^{[3]}) \rightarrow \text{final o/p}$$

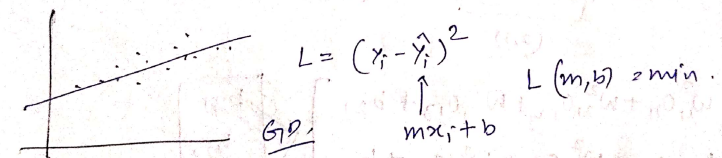
$$\sigma \left(\left(\sigma(a^{[0]}w^{[1]} + b^{[1]}) \right) w^{[2]} + b^{[2]} \right) w^{[3]} + b^{[3]}$$

18/07/25

What is Loss fn?

Loss fn is a method of evaluating how well your algorithm is modelling your dataset.

high $\rightarrow$ poor
small $\rightarrow$ great
L (parameters) $\leftarrow$ $f(x) = wx_1 + 2$



Why is Loss fn important?

[You can't improve what you can't measure]

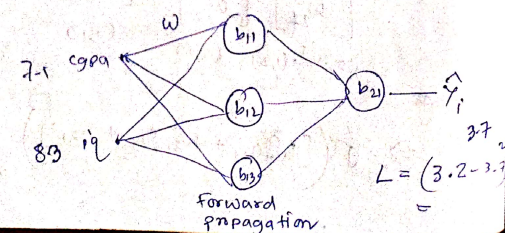
Peter Drucker

loss function $\rightarrow$ eye of ML algorithm

loss fn in Deep Learning?

cpu iq package (lpa)

7.1	83	0.2
8.5	91	4.5
6.3	102	6.1
5.1	89	2.7



not (1/r) → forward Propagation → $\hat{y}_{i1}$ → Loss → determine new value of w & b using Gradient Descent

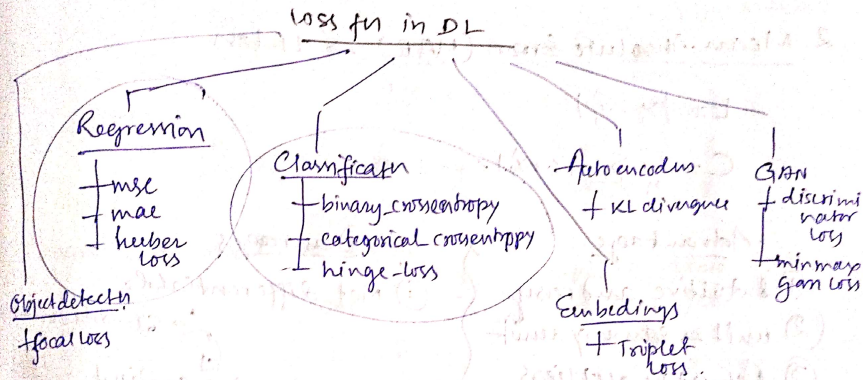
↓

GD: new w, b → Loss → $\hat{y}_{i2}$ → Forward Prop. → new 2 (1/r)

brain will get minimum

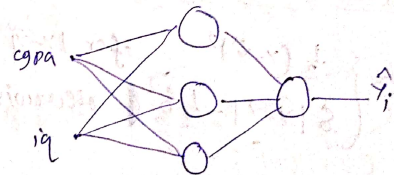
if epochs = 10
8 no. of rows = 5
or inputs

total $10 \times 5 = 50$ updates in w & b



Loss fn VS Cost Fn

cgpa	iq	package	predn
6.3	100	6.3	6.1
7.1	91	4.1	4
8.5	83	3.5	3.7
9.2	102	7.2	7



Loss fn → single training eg.
Error $y_1 = 6.3, \hat{y}_1 = 6.1$
 $(y_1 - \hat{y}_1)^2$

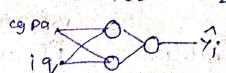
Cost fn → for whole data
 $\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$
 $= \frac{1}{4} \sum_{i=1}^4 (y_i - \hat{y}_i)^2$
 $= \frac{1}{4} ((6.3 - 6.1)^2 + (7.1 - 4)^2 + (8.5 - 3.7)^2 + (9.2 - 7)^2)$

Mean Squared Error (MSE)

→ squared loss, L^2 loss.

$$(y_i - \hat{y}_i)^2$$

cgpa	iq	y_i	$\hat{y}_i$
6.3	100	6.3	6.1
7.1	91	4.1	4
8.5	83	3.5	3.7
9.2	102	7.2	7

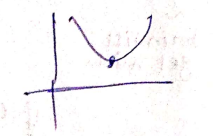
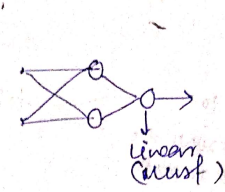
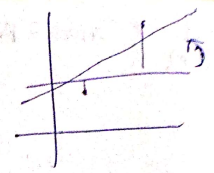


USB Advantage

- 1 Easy to interpret
- 2 Differentiable
- 3 1 Local Minima

Disadvantage

- 1 Error unit → squared
- 2 Robust to outliers



2. Mean Absolute Error (MAE) → L1 Loss

$$L = |y_i - \hat{y}_i|$$

$$C = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Advantage

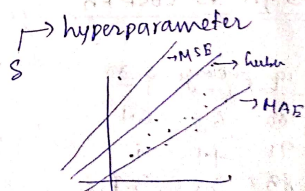
- 1 Intuitive and easy
- 2 unit → same y unit
- 3 Robust to outliers

Disadvantages

- 1 Not differentiable
- GD → differentiable
Sub Gradient

2. Huber Loss

$$L = \begin{cases} \frac{1}{2} (y - \hat{y})^2 & \text{for } |y - \hat{y}| \leq \delta \\ \delta |y - \hat{y}| - \frac{1}{2} \delta^2 & \text{otherwise} \end{cases}$$



4. Binary Cross Entropy (log-loss)

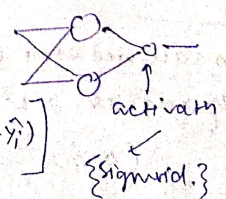
→ classification
→ 2 classes

$$\text{loss fn} = -y \log(\hat{y}) - (1-y) \log(1-\hat{y})$$

cgpa	iq	placement
8	80	1
7	70	0
6	60	0

y → actual
 $\hat{y}$ → prediction

$$\text{cost fn} = -\frac{1}{n} \left[\sum_{i=1}^n y_i \log(\hat{y}_i) + (1 - \hat{y}_i) \log(1 - \hat{y}_i) \right]$$



Advantage

- 1 Differentiable

Disadvantage

- 1 Multiple Local Minima
- 2 Not intuitive

5. Categorical Cross Entropy [used in Softmax Regression]

→ [Multi-class] classification

1 point →
$$L = - \sum_{j=1}^K y_j \log(\hat{y}_j)$$

(K = no. of classes)

$$L = -y_1 \log(\hat{y}_1) - y_2 \log(\hat{y}_2) - y_3 \log(\hat{y}_3)$$

for yes.

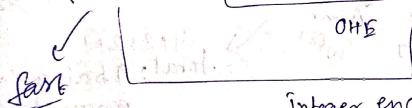
$$f(z) = \frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

Activation
neurons
class ← categories

for no.

$$\frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}} \quad \text{for Maybe} \rightarrow \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}$$

Sparse categorical cross entropy



Integer encode.

[0.1, 0.4, 0.5] → 1.

Suppose $(y_1, y_2, y_3 = 1)$

① → (7, 70) → $-1 \times \log(0.1)$

[0.1, 0.4, 0.6] → 2 → $L \rightarrow (8, 80)$
 $-1 \times \log(0.4)$

[0.1, 0.4, 0.6] → 3 → $L \rightarrow (6, 60)$
 $-1 \times \log(0.6)$

$$L = - \sum_{j=1}^K y_j \log(\hat{y}_j) \rightarrow C = - \frac{1}{n} \sum_{j=1}^n \sum_{i=1}^K x_{ij} \log(\hat{y}_{ij})$$

Summary

Key → MSE
 → outliers → MAE
 → Huber loss

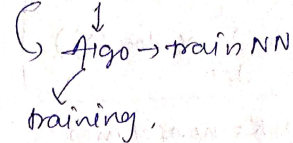
Classification → binary → Binary class classification (BCE)
 → Multi → few cat → Categorical Cross Entropy (CCE)
 → many " → (SCE) → sparse

Backpropagation

Prerequisites

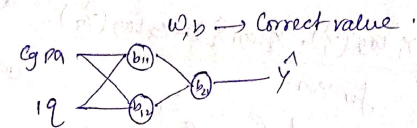
→ Gradient descent
 → forward propagation

Backprop



cgpa	iq	LPA
8	80	8
7	70	7

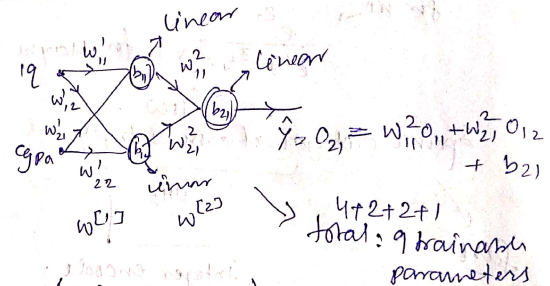
reg.



backprop.

iq	cgpa	LPA
80	8	3
60	9	5
70	5	8
120	7	11

Regression



Steps: 0) init w, b (let w=1, b=0)

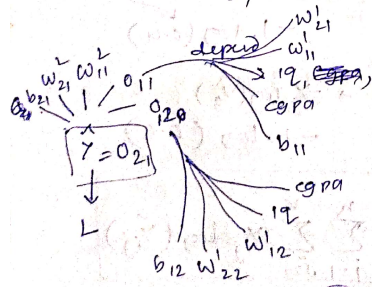
1) you select a pt. (row)

2) Predict (LPA) → forward propagation [Dot product]

3) choose a loss fn. → MSE

4) w, b - update

→ Gradient descent



$$w_{new} = w_{old} - \eta \frac{\partial L}{\partial w_{old}}$$

$$b_{new} = b_{old} - \eta \frac{\partial L}{\partial b_{old}}$$

$$w_{11}^2_{new} = w_{11}^2_{old} - \eta \left(\frac{\partial L}{\partial w_{11}^2_{old}} \right)$$

derivative of loss

$$w_{21}^2_{new} = w_{21}^2_{old} - \eta \frac{\partial L}{\partial w_{21}^2_{old}}$$

→ 9 trainable parameters

$$b_{21}_{new} = b_{21}_{old} - \eta \frac{\partial L}{\partial b_{21}_{old}}$$

→ 9 derivatives ✓